The `--version` flag prints out the Django version and exits. It isn't clear to me why they bother, as there is a `version` management command that does the same thing.

The `--verbosity` flag gets used by the built-in Django management commands to display more or less debug information as part of their output. Currently, using it with the `musicians` command does nothing, but `options["verbosity"]` is populated by the flag, so you could change your output based on its value.

The `-settings` flag changes which configuration Django uses, with the default being your settings.py file. Similarly, you can change your Python path if you need to add additional modules, using `--pythonpath`.

By default, raising a `CommandError` does not display a stack trace; instead, it prints a colorized error message. If you want to see the stack trace, the `--traceback` flag outputs it if an exception occurs.

Django automatically applies terminal colors to your management command if your terminal supports them. You can turn this off using `--no-color`, or if you're using the command in a situation where color is normally off (e.g., piping your command to another command), `--force-color` forces coloring to be turned on.

Pretty pretty colors

In addition to controlling your command's colors through the `--no-color` and `--force-color` flags, you can also change the colors themselves. This isn't done through the code but through an environment variable: `DJANGO_COLORS`. Django ships with three different palettes: dark, light, and nocolor. On a Unix-based machine using a bash-compatible shell, you change to the light palette with the following command:

```
(venv) RiffMates$ export DJANGO_COLORS="light"
```

This shell variable even allows you to control exactly which colors to use. For more information on controlling management command colors, see http://mng.bz/eEqw.

The same system checks that get run when the development server gets started are also available through the `check` management command. These checks also get run by default when you invoke your own commands. To skip them, use the `--skip-checks` flag. Alternatively, you can set your command class's `requires_system_checks` attribute to an empty `tuple` to disable checking programmatically.

9.4 Testing, and using STDOUT and STDERR

Chapter 8 was all about testing Django views; your custom management commands are code, so they deserve to be tested as well. To help with this, Django provides the `django.core.management.call_command` function, allowing you to invoke a management command from code.

A management command is often all about its output. For example, the `musicians` command displays musician information to the screen. Recall that I insisted you use `self.stdout.write()` instead of calling `print()`; testing is why. The `BaseCommand` class provides proxies to both STDOUT and STDERR, which the `call_command()`